



**Tecnológico
de Monterrey**

Estructura de Datos y Algoritmos
Fundamentales

Prof. David Alonso Cantú Delgado

Reflexión

Javier Pineda Ponce - A00845219

● Importancia de ordenar

La importancia real de ordenar los logs es para auditar cronológicamente los sucesos que pasaron conforme al tiempo en un ataque o intento del mismo. Esto en ciberseguridad es muy útil para los analistas de respuesta en los incidentes y para la investigación forense digital para poder recrear y estudiar el vector de ataque para que los ingenieros puedan reforzar el sistema de seguridad. Para los incidentes responders el tiempo es crítico, y tener los logs ya ordenados es vital para que puedan ver por dónde están atacando el sistema y poder contener el ataque lo antes posible.

En otro caso fuera del área de ciberseguridad es el la auditoría de mensajes o acciones que se realizan en un servidor. Por ejemplo si se cambian los permisos ver quien y que se cambio es importante para que no todo sea anónimo y haya un registro de cuando se hizo. Algo similar sucedió en la vida real cuando en un repositorio open source un usuario empezó a meter código malicioso para intentar hackear casi todos los servidores con base linux, sin los logs ordenados hubiera sido un caos ver desde cuando se unió y que aportaciones hizo al repositorio. Normalmente estos logs ya vienen ordenados, pero en dado caso que no estén tenerlos en orden siempre es importante para buscar fechas rápidamente y poder reconstruir lo que sucedió en aquel momento.

● Comparación

Los más lentos teóricamente como Bubble Swap e Insertion son muy fáciles de implementar, pero mientras más datos tengan que manejar se vuelven más ineficientes.

Selection aunque igual es teóricamente lento es una excepción a la regla, ya que utiliza de manera más eficiente la memoria en esta situación en concreto, haciéndola que sea mucho más rápido que los otros algoritmos y destacando.

Merge es perfecto porque es muy consistente, ya que no le importa el tipo de dato que esté viendo siempre dividirá estos a la mitad. Su única debilidad es que si los datos son demasiados podría tener problemas de memoria haciendo que pierda su ventaja al ser muy rápido.

Quick suele ser siempre el más rápido como lo dice por su nombre, pero su debilidad con el archivo casi ordenado donde se topa con muchos datos duplicados es donde empieza a tropezar. Sus condiciones no manejan muy bien los datos duplicados, lo que causa un

desbalance en sus divisiones y convirtiéndolo más lento de lo que es con datos completamente desordenados.

Shell es la mejora de Insertion, es como tener dos algoritmos funcionando en uno mismo, lo que hace especial shell es semi acomodar los datos para que cuando ya estén casi acomodados utilice el método de Insertion para terminar de acomodar los datos perfectamente. Su única debilidad es que puede ser inestable en algunas ocasiones, si el algoritmo cae en un log que tiene exactamente la misma hora podría invertir el orden sin darse cuenta.

● Interpretación

Mi predicción es que no pensaba que los algoritmos podrían ocupar demasiada memoria al grado de que algunos algoritmos que resultan ser teóricamente más rápidos puedan terminar siendo mucho más lentos que un algoritmo teóricamente más lento.

Mi predicción era que todos los algoritmos se comportan a su nivel, el más lento siendo el más lento y el más rápido siendo el más rápido, pero al hacer este proyecto me di cuenta que los datos también importan. El algoritmo que decidamos usar para ordenar debe de ser el adecuado para el tipo de datos que estará manejando, no hay algoritmo perfecto, esto dependerá de cuanta memoria queremos utilizar y el los datos. Esto me di cuenta cuando QuickSort fue mucho más lento con un archivo casi ordenado, al principio pensé que era un error de mi programa pero cuando investigué el porque comprendí que era por los datos, se confunde con los datos duplicados y baja su nivel por eso mismo.

● Uso de la IA

En gran parte, siendo realistas, este proyecto me hubiera tomado mucho más tiempo sin la IA. Me ayudó demasiado con preguntas y especialmente la implementación para leer los archivos ya que es algo con lo que nunca había trabajado antes. La IA si tuvo errores especialmente con la lectura de archivos que luego yo tuve que ver por mi cuenta para arreglarlo. La implementación de algoritmos fue muy rápida porque al solo abrir el archivo donde están los algoritmos que tenía que implementar copilot básicamente ya casi que sabía como tenía pensando en hacer la estructura de mi código, lo que me ahorró muchísimo tiempo ese día. Donde empecé a tener problemas con la IA fue en la misma parte de la implementación para leer el archivo, ya que como no sabia como hacerlo yo mismo la IA hizo la mayor parte pero como el archivo no estaba en el mismo directorio dude e intente correrlo, y lo que me

esperaba era que no se leyera el archivo porque nunca le digo al programa en donde esta el archivo que le solicitó leer. Entonces fue allí donde empecé a hacer preguntas y buscar la manera de corregir ese error. Aquí están los prompts que iniciaron todo eso:

“el programa se cierra solo al intentar cargar el archivo log607-2

no se lee el archivo porque esta en otro directorio”

“out_of_range con substr

tengo este error que no lee la hora.”

Inicialmente, la IA me recomendó usar un método de lectura rígido (`substr(6, 2)`) asumiendo ineficientemente que todos los logs tendrían exactamente la misma longitud de caracteres en la hora. Cuando vi que el programa empezó a arrojar errores después de unos momentos de elegir el archivo es cuando pregunte por el error, me lo describió y vi los datos de logs para confirmar el error, a veces agarraba el año como si fuera la hora. Al final le puse que se fijara que después de leer había unos dos puntos (:), si hay esto confirmaba que es la hora y si no había confirmaba que es el año. Se eliminó la lectura rígida y me quedé con este método dinámico que es mucho más consistente.

El uso de IA hace que la programación sea mucho más accesible y eficiente revisando constantemente errores y preguntando por errores que no conoces, pero todo el código que te sugiere implementar tiene que ser revisado críticamente para ver si realmente no mal interpreto lo que se está haciendo o vice versa.